

COE 311K Final Project

[Link to my GitHub](#)

Part 1 - Numerical Solution of Second-Order ODEs

Section 1: Introduction & Model System

This project focuses on a spring-mass-damper (SMD) system, similar to what is used in vehicle suspension systems. In this setup, a mass is connected to a spring and a damper, and when the system is disturbed, it responds with oscillations that gradually decrease over time due to damping.

For a spring-mass-damper system, the motion can be described by the differential equation $m\ddot{y} + c\dot{y} + ky = 0$ where m is the mass (kg), c is the damping coefficient (N*s/m), and k is the spring constant (N/m). The variable y represents the displacement from equilibrium (m). Rewriting the equation to isolate the acceleration gives: $\ddot{y} = -c/m(\dot{y}) - k/m(y) = 0$. This form shows how both damping and spring force contribute to the system's acceleration.

Let $y_1 = y$ and $y_2 = \dot{y}$. The system then becomes $\dot{y}_1 = y_2$ and $\dot{y}_2 = -c/m(y_2) - k/m(y_1)$.

Section 2: Parameter Research & Justification

For this project, I am looking at a real-world case involving the suspension system of a 2019 Nissan Altima 2.5 SR (without AWD) as it drives over a typical neighborhood speed bump at a low speed. The car has a curb weight of about 3290 lb (1492 kg), the speed bump height is approximately 3 inches (0.0762 m), and the vehicle is traveling at roughly 10 mph (16.09 kph).

Parameter Values:

Parameter	Symbol	Value	Units	Source
Mass per spring	m	375	kg	Altima curb weight divided by 4
Spring constant	k	35,000	N/m	MacPherson strut specification based on manufacturer data from Nissan documentation
Damping ratio	ζ	0.36	—	MacPherson strut specification based on manufacturer data from Nissan documentation
Initial displacement	y_0	0.0762	m	Standard height of a residential speed bump
Vehicle velocity	v_{car}	16.1	kph	Based on an assumed scenario

Tire radius	r	0.337	m	Tire diameter expressed as radius in meters
Simulation time	t	3.0	s	Sufficient time for the system to settle back to equilibrium

Derived Parameters:

Parameter	Symbol	Formula	Value	Units
Natural frequency	ω_n	$\sqrt{\frac{k}{m}}$	4.49	rad/s
Damped frequency	ω_d	$\omega_n \sqrt{1 - \zeta^2}$	4.26	rad/s
Damping coefficient	c	$2\zeta\sqrt{mk}$	4282.00	N·s/m
Initial vertical velocity	$\dot{y}_0$	$v_{car} \cdot \sin(\arctan(\frac{y_0}{r}))$	3.58	kph

Sources:

1. <https://www.cars.com/research/nissan-altima-2023/specs/>
2. <https://www.nissanusa.com/vehicles/cars/altima/specs-trim.html>
3. <https://www.edmunds.com/nissan/altima/2023/features-specs/>
4. <https://www.caranddriver.com/nissan/altima-2023>
5. <https://highways.dot.gov/safety/speed-management/traffic-calming-eprimer/module-3-part-2#3.10>
6. <https://www.tirerack.com/>

Section 3: Numerical Implementation

Both numerical methods are applied to the same governing function f to ensure a consistent and fair comparison between the models. The damping coefficient c is determined from a given damping ratio obtained from reliable sources. The initial vertical velocity of 3.59 kph is estimated using a physically based approximation that accounts for the vehicle's speed, the height of the speed bump, and the tire radius, which is appropriate for low-speed conditions and standard bump geometries.

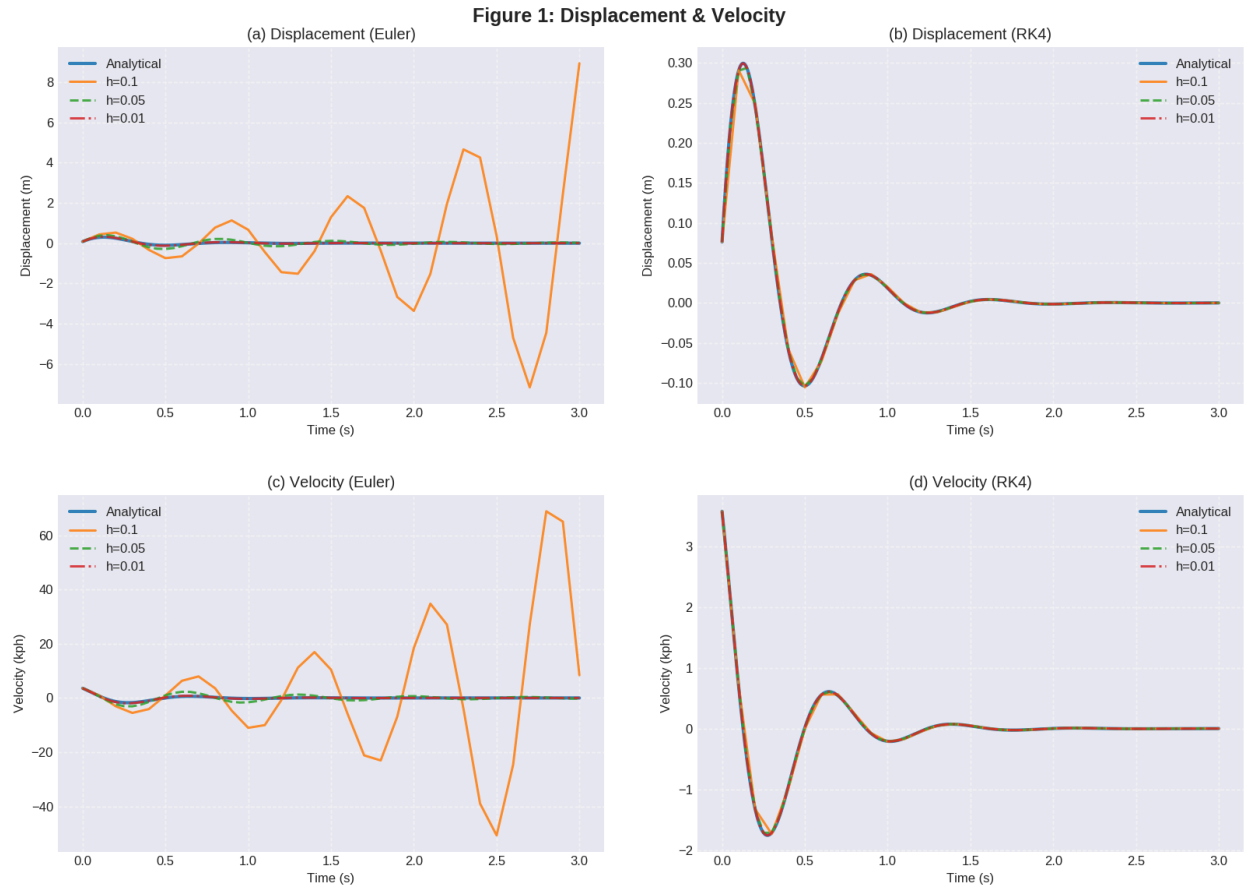


Figure 1 shows displacement and velocity plots generated using Euler's Forward method and the RK4 method with varying step sizes.

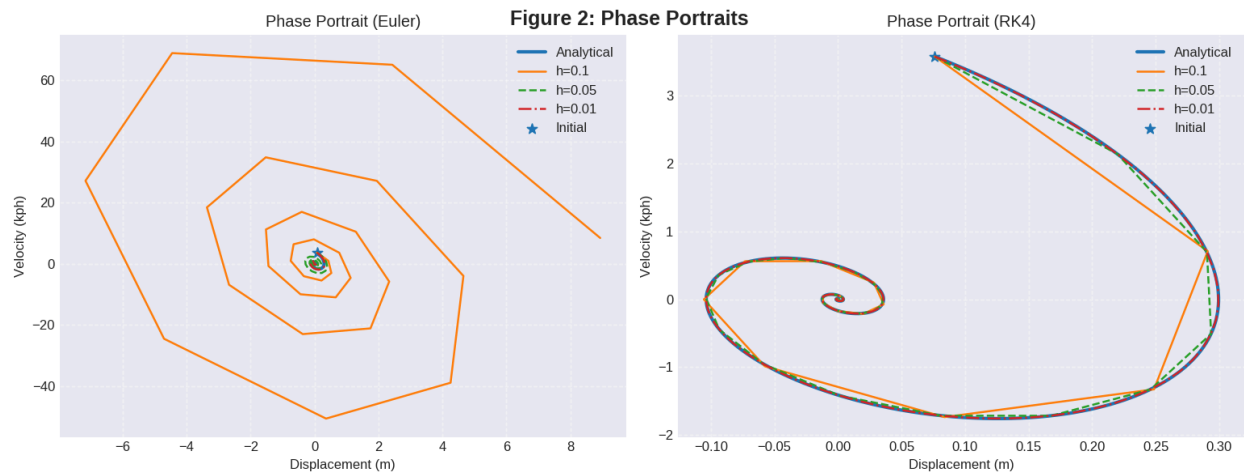


Figure 2 shows phase portraits generated using Euler's Forward and RK4 methods, with an initial displacement of 0.0762 m and an initial velocity of 3.58 kph.

Section 4: Solutions & Comparisons

Figure 1 represents the suspension response as the vehicle travels over a speed bump at approximately 16.1 kph. The displacement initially rises slightly above the bump height because the vehicle's momentum carries it beyond the equilibrium position before the suspension fully reacts. Since the system is underdamped, the response exhibits oscillations that gradually decrease in amplitude before settling back to equilibrium.

Across all step sizes, the RK4 method consistently follows the expected physical behavior more accurately than Euler's Forward method. At a larger step size ($h = 0.1$), Euler's method becomes noticeably unstable and introduces significant numerical error, while reducing the step size improves its stability and accuracy. However, RK4 remains closely aligned with the system's true response regardless of step size.

Figure 2 illustrates the phase portrait behavior, where the system trajectory spirals inward toward equilibrium. The RK4 solution closely matches the smooth inward spiral predicted by the analytical model, while Euler's method shows noticeable deviation at larger step sizes. In particular, at $h = 0.1$, Euler's method shows artificial energy into the system due to numerical error, causing a slight outward spiral before eventually converging as the step size lowers.

Section 5: Stability Analysis

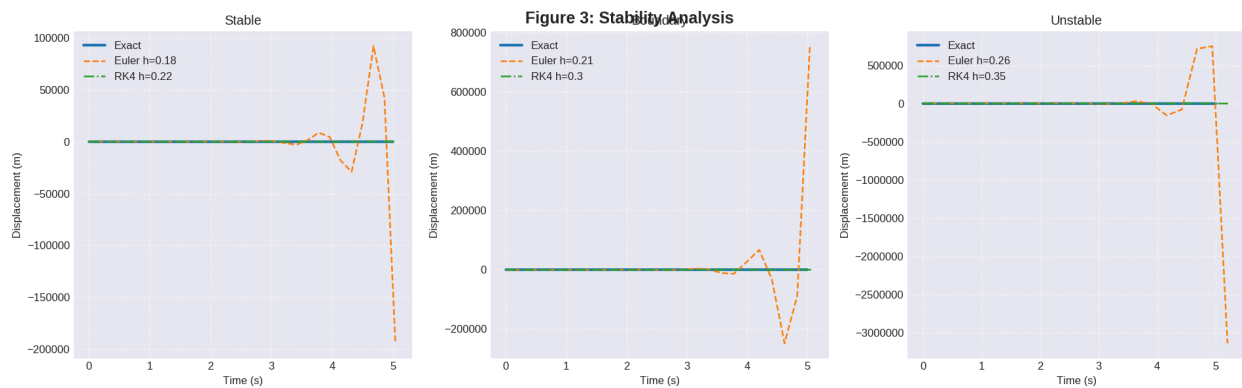


Figure 3 shows a stability comparison between Euler's method and RK4 at step sizes near their respective stability limits (Euler: 0.224, RK4: 0.32).

The stability limit for Euler's method is around $h = 0.22$, while RK4 remains stable up to about $h = 0.3$. In both cases, the numerical solutions remain stable when the step size is below their respective thresholds. However, once these limits are exceeded, the solutions become unstable and therefore end up separating.

Compared to Euler’s method, RK4 can handle larger step sizes before instability occurs, as shown in Figure 3. When instability appears, the model produces increasingly large oscillations, which are non-physical and do not represent real vehicle behavior.

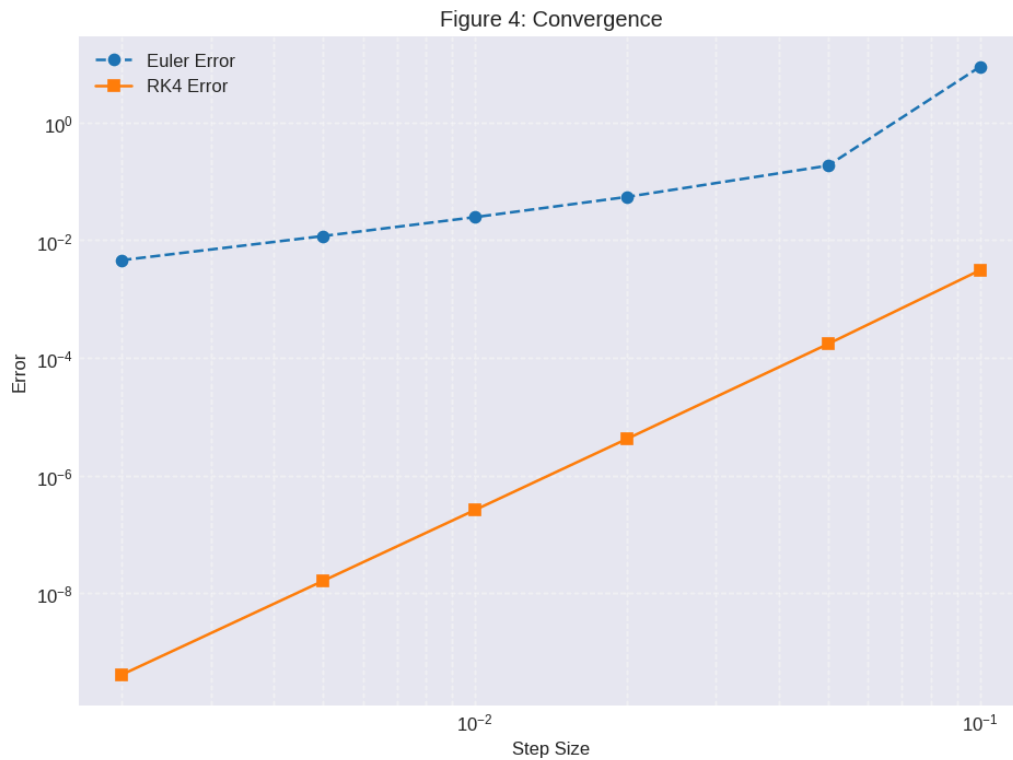


Figure 4 shows a log-log plot of error versus step size, illustrating the convergence behavior of both numerical methods compared against the theoretical reference slopes $O(h)$ and $O(h^4)$.

Euler’s method exhibits first-order convergence, meaning that reducing the step size by half approximately reduces the error by a factor of 2. In contrast, RK4 has fourth-order convergence, so halving the step size reduces the error by roughly 16 times. This behavior is reflected in the measured slopes down in Figure 5, which closely align with the expected theoretical convergence rates for both methods.

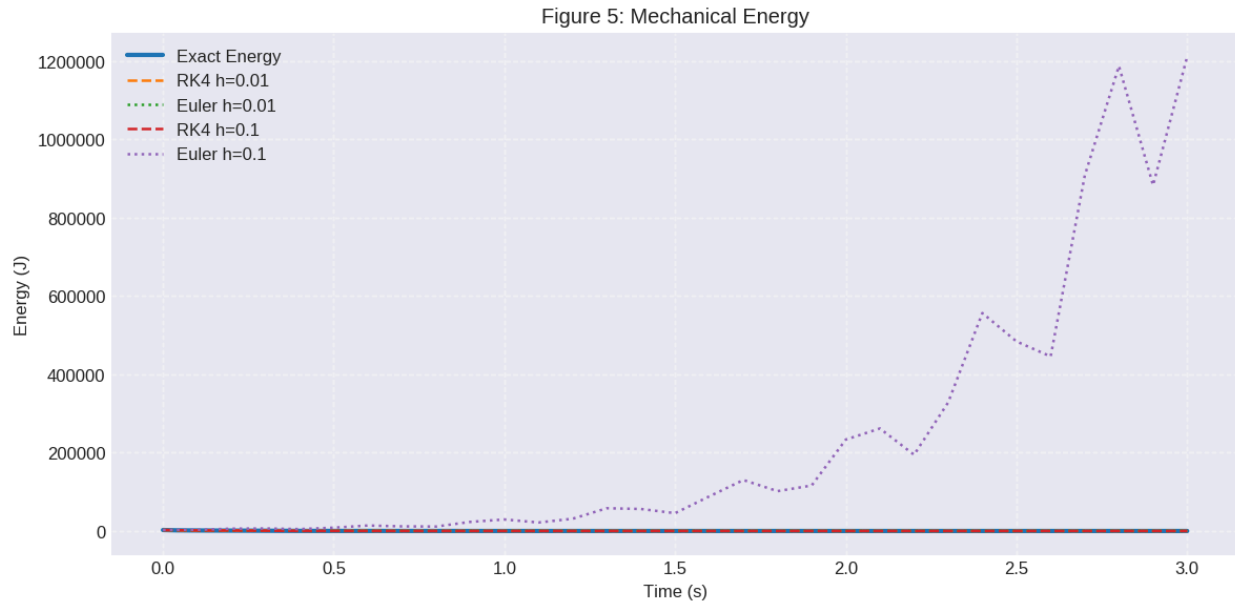


Figure 5. Total mechanical energy of the system as a function of time, showing a gradual decrease due to damping effects

The total mechanical energy of the system could steadily decrease over time due to damping, since the shock absorber dissipates energy rather than adding to it. However, at a larger step size, Euler's method shows an artificial increase in energy at each step. This occurs because small numerical errors accumulate and effectively introduce energy into the system. In contrast, the RK4 method closely follows the expected energy decay, remaining consistent with the physical behavior of the system throughout the simulation.

Section 6: Conclusion

RK4 performed noticeably better than Euler's Forward method across all the step sizes I tested. From Figure 2, the RK4 phase portraits consistently spiral inward toward the origin, which matches the expected physical behavior. In contrast, Euler's method becomes unstable at $h = 0.1$ and still shows some drift even at smaller step sizes.

Although RK4 requires four function evaluations per step compared to just one for Euler's method, it makes up for this by allowing much larger step sizes while maintaining accuracy. In practice, RK4 can achieve similar accuracy to Euler's method at about ten times the step size, which can actually make it more efficient. For example, in the 3-second simulation, RK4 with $h = 0.01$ gives better results than Euler's method with $h = 0.001$.

Based on these results, RK4 with a step size around $h = 0.01$ is a good choice for modeling systems like this. Euler's method is not ideal for oscillatory systems because numerical errors can introduce artificial energy, which leads to misleading physical behavior. While Euler's method can still work with very small step sizes, RK4 is generally the better option.

One key takeaway is the importance of using realistic and consistent parameters that match the physical system being modeled. Looking at energy trends also helps explain what is happening beyond just the plots, especially when identifying issues like artificial energy gain in the method. Another important point is that stability alone does not guarantee accuracy.

Part 2: Stiff ODEs and Implicit Methods

Section 1: Introduction and System Selection

In this project, I studied a stiff first-order differential equation that models a system where a state variable rapidly relaxes toward a slowly changing equilibrium. These types of systems appear in engineering problems where fast internal dynamics interact with slow external forcing. The governing equation is: $dC/dt = -k(C - C_{eq}(t))$ where $k = 10^4 \text{s}^{-1}$, $C(0) = 0$, $C_{eq}(t) = 1 + 0.1 \sin(0.1t)$, and simulation time: $t \in [0, 60] \text{s}$. This system represents a quantity C that rapidly adjusts toward a slowly oscillating equilibrium. The equilibrium changes gradually compared to how fast the system reacts.

The model behaves like a fast-adjusting process driven by a slowly varying input. Even though the equilibrium changes over seconds, the system itself responds on the order of milliseconds, which is what creates stiffness. This type of behavior shows up in fast chemical reactions, thermal equilibration problems, and signal tracking systems.

Because the equation is linear, it has a closed form solution: $C(t) = C_h(t) + C_p(t)$ where the homogeneous part decays really quickly due to e^{-kt} , and the particular solution closely follows $C_{eq}(t)$ after the initial transient dies out. Since k is very large, the solution essentially “locks onto” the equilibrium almost right away.

Section 2: Demonstrating Stiffness

To demonstrate stiffness, I first applied Euler’s forward method with different step sizes. When using a reasonably small step size, the solution stays stable and follows the expected behavior. However, when the step size is increased to something like $h = 0.1$, the numerical solution becomes unstable and diverges rapidly. This shows that stability is really sensitive to step size, even though the actual physical system is smooth.

The stiffness comes from the value of $k = 10^4$, which introduces a very fast time scale compared to the slow oscillation in $C_{eq}(t)$. The fast time scale is approximately: $\tau \sim 1/k = 10^{-4}$ while the forcing changes on the order of: $\tau \sim 10$ this gives a stiffness ratio of roughly: $\tau_{\text{slow}} / \tau_{\text{fast}} \approx 10^5$.

For stability, Euler Forward requires: $h < 2/k \approx 2 \times 10^{-4}$. This means that to simulate 60 seconds we have to meet two standards: 300,000+ steps and each run must become more computationally expensive. If we try to use large step sizes for efficiency, the solution becomes unstable. This confirms that explicit methods are not practical for this system.

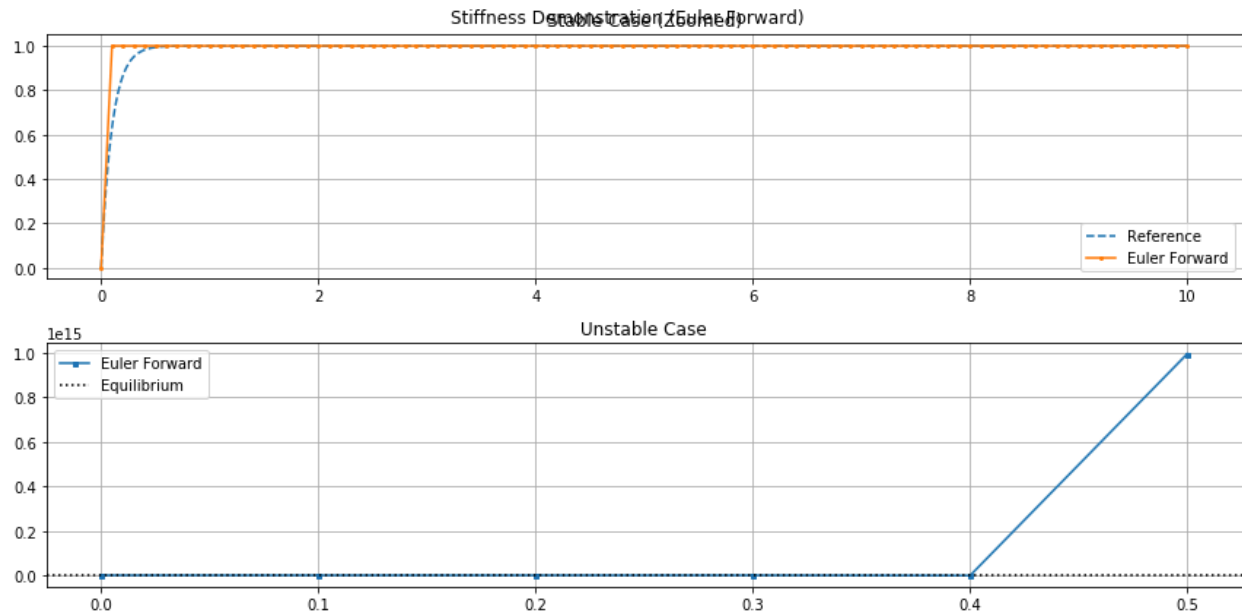


Figure 1 shows that Euler's Forward method with $h=0.0001$ (top) remains only marginally stable, while the standard Euler's Forward result (bottom) exhibits severe oscillations and instability.

Section 3: Mathematical Setup

To solve this stiffness issue, I used Euler's Backward method: $C_{n+1} = C_n + hf(t_{n+1}, C_{n+1})$ substituting the governing equation: $C_{n+1} = C_n - hk(C_{n+1} - C_{eq}(t_{n+1}))$ this leads to a nonlinear equation that must be solved at every time step.

I rewrote the problem as: $G(C) = C - C_n + hk(C - C_{eq}(t_{n+1}))$ which is simplified as $G(C) = (1 + hk)C - C_n - hkC_{eq}(t_{n+1})$

To solve $G(C) = 0$, I used Newton-Raphson iteration: $C^{(i+1)} = C^{(i)} - G(C^{(i)})/dG/dC$ where $dG/dC = 1 + hk$. Because the derivative is constant, the system is effectively linear in C , meaning Newton-Raphson converges extremely quickly (usually in one iteration per step). The initial guess was taken from Euler Forward: $C^{(0)} = C_n + hf(t_n, C_n)$.

Section 4: Implementation

```
k_val    = 1e4
C_init    = 0.0
t_final   = 60.0

def C_eq(t):
    return 1.0 + 0.1 * np.sin(0.1 * t)

def dCeq_dt(t):
    return 0.01 * np.cos(0.1 * t)

def f(t, C):
    return -k_val * (C - C_eq(t))
```

```
def newton_raphson(t_next, C_n, h, tol=1e-10, max_iter=20):

    dGdC = 1.0 + h * k_val
    C = C_n + h * f(t_next - h, C_n)
    iterations = 0

    for _ in range(max_iter):
        G = C - C_n - h * f(t_next, C)
        C_new = C - G / dGdC
        iterations += 1

        if abs(C_new - C) < tol:
            return C_new, iterations

    C = C_new

    return C, iterations
```

```

def euler_backward(h, T=t_final, C0=C_init):
    t = np.arange(0, T + h, h)
    C = np.zeros(len(t))
    C[0] = C0
    iters = []

    for n in range(len(t) - 1):
        C[n+1], it = newton_raphson(t[n+1], C[n], h)
        iters.append(it)

    return t, C, np.array(iters)

fig1 = plt.figure(figsize=(12, 6))
gs1 = fig1.add_gridspec(2, 1)

```

Section 5: Results & Analysis

Euler's Backward method produces stable solutions across all tested step sizes. Even with large step sizes, the solution remains accurate and does not exhibit instability. This is a major improvement compared to Euler Forward, which fails at large step sizes. As the step size decrease, the solution will converge toward the analytical side, but even coarse steps already produce good accuracy.

The Newton-Raphson solver converges extremely quickly for this system. In most cases, it only takes 1-2 iterations per time step, confirming that the system behaves almost linearly with respect to the unknown variable at each step.

The global error decrease approximately linearly with step size, which is expected since Euler's Backward method is first-order accurate. On a log scale, the error follows an approximately $O(h)$ trend, which affirms the theoretical expectations.

Section 6: Performance Analysis

Method	Step Size	Steps	Stability	Observations
Euler's Forward	0.1	600	Unstable	Diverges
Euler's Forward	0.0001	600,000	Stable	Very expensive

Euler's Backward	0.1	600	Stable	Accurate
Euler's Backward	0.01	6,000	Stable	Good balance
Euler's Backward	0.001	60,000	Stable	High accuracy

Implicit Euler dramatically reduces computational constraints while maintaining stability, which makes it much more suitable for stiff systems.

Section 7: Conclusions

This project demonstrated why stiffness is fundamentally a stability issue rather than what is normally perceived (accuracy).

Main conclusions:

- The system is stiff due to a large separation of time scales
- Euler Backward remains stable even for larger step sizes
- Euler Forward fails unless extremely small step sizes are being used
- Implicit methods are essential for stiff engineering systems
- Newton-Raphson converges very quickly because the system is effectively linear

In real applications, stiffness determines the choice of numerical method more than accuracy requirements and standards. Explicit methods become impractical, while implicit methods provide stable and efficient solutions.

Part 3: Damping and Adaptive Step Sizes

Section 1: Review of Part 2

In Part 2, I worked with a stiff first-order ODE that models a fast relaxation process toward a slowly equilibrium. The governing equation was: $dC/dt = -10^4(C - C_{eq}(t))$ where the equilibrium is defined as: $C_{eq}(t) = 1 + 0.1\sin(0.1t)$. This system is stiff because the reaction rate constant is extremely large, meaning the solution reacts almost instantly to changes in equilibrium. The stiffness ratio is on the order of 10^5 , which makes explicit methods like Euler's Forward unstable unless the step size is extremely small.

From Part 2, I used Euler's Backward with Newton-Raphson to handle stability. Even though the method is stable for larger time steps, the accuracy and convergence still depend heavily on how Newton's method behaves inside each step. I also defined a reference solution using the quasi-steady approximation: $C(t) \approx C_{eq}(t) - 1/k(dC_{eq}/dt)$. This gave me a baseline to compare numerical solutions against.

Section 2: Newton-Raphson Damping Implementatin

When using Newton-Raphson to solve the implicit Euler step, the update can fail if the initial guess is too far from the root. This happens more often when the step size is large, the system is stiff, or the residual is highly sensitive. In these cases, the standard Newton update is $C^{k+1} = C^k - G(C^k)/G'(C^k)$ can overshoot and actually increase the residual instead of reducing it.

To stabilize this, I implemented a damping factor: $C^{k+1} = C^k + \alpha \Delta C$ where $\Delta C = -G(C^k)/G'(C^k)$. Instead of always taking the full Newton step, I used a line search strategy: start with $\alpha = 1$, check if residual decrease, if not halve α , repeat until improvement. This ensures every update is actually moving toward convergence instead of destabilizing the iteration.

From my plots, I compared standard Newton vs. damped Newton at large step sizes ($dt = 2$ and $dt = 5$): standard Newton can occasionally converge slowly or become unstable, damped Newton consistently reduces the residual, and damping mainly activates in the early iterations when the guess is poor. So in my case, damping acts more like a safety mechanism than something constantly used.

```

def damped_residual_history(c_now, t_next, dt, max_iter=15):
    c_guess = c_now + dt * ode_rhs(t_next - dt, c_now)
    hist = [abs(residual(c_guess, c_now, t_next, dt))]
    alpha_hist = []

    for _ in range(max_iter):
        r = residual(c_guess, c_now, t_next, dt)
        if abs(r) < 1e-10:
            break

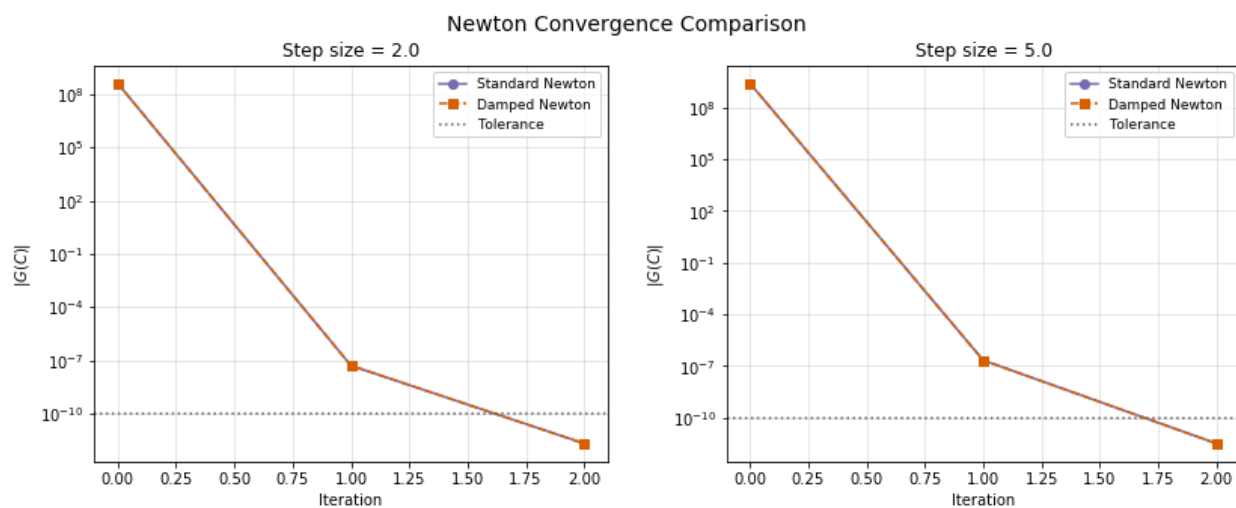
        step = -r / residual_prime(dt)
        alpha = 1.0
        current_res = abs(r)

        for __ in range(20):
            trial = c_guess + alpha * step
            if abs(residual(trial, c_now, t_next, dt)) < current_res:
                break
            alpha *= 0.5
            if alpha < 1e-4:
                break

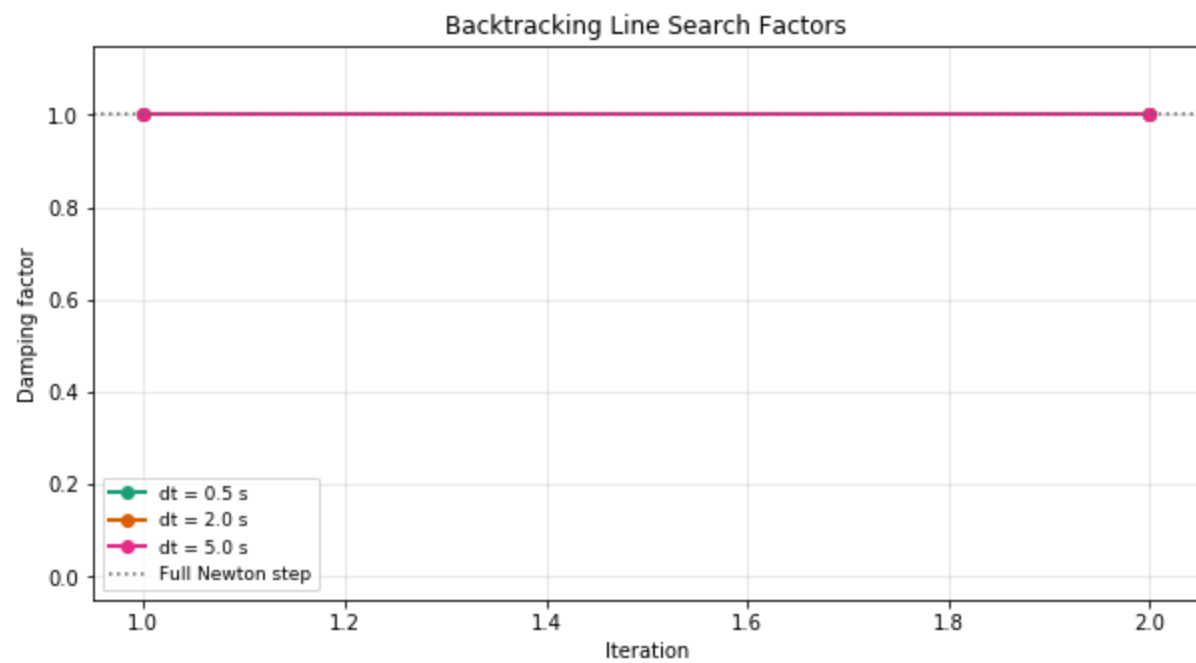
        alpha_hist.append(alpha)
        c_guess = c_guess + alpha * step
        hist.append(abs(residual(c_guess, c_now, t_next, dt)))

    return hist, alpha_hist

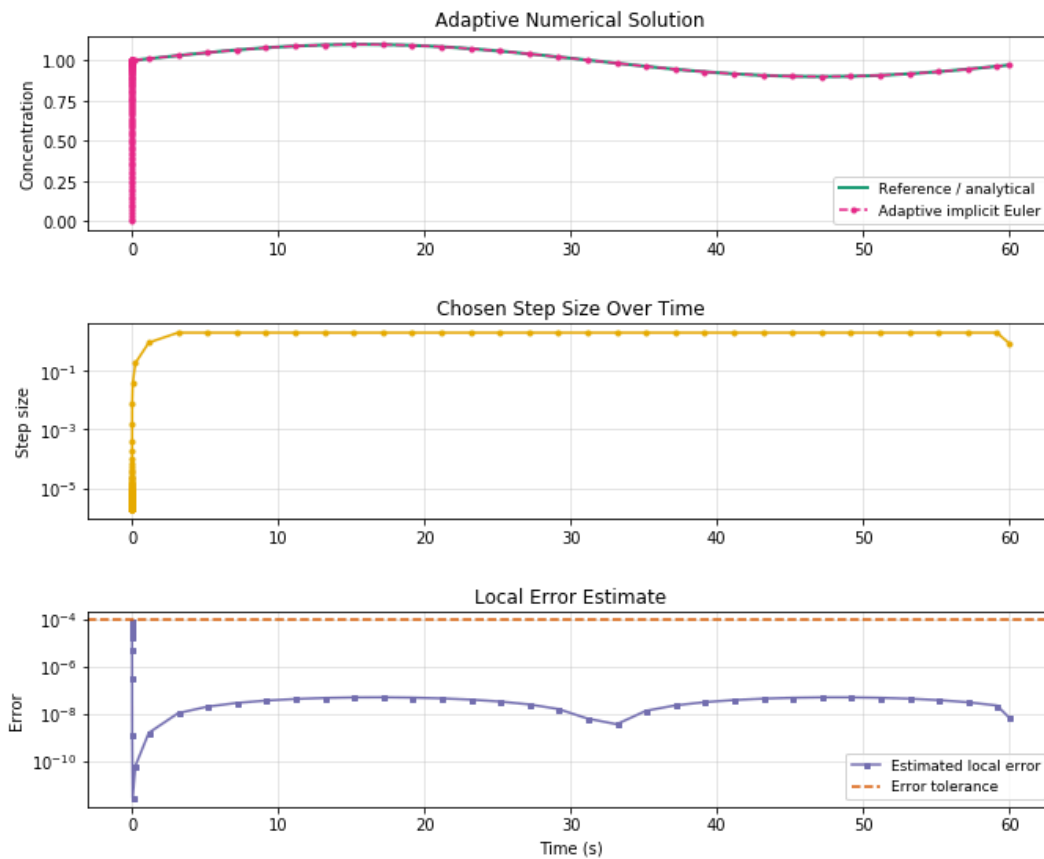
```



There is no difference between the damped and undamped NR convergence. Both of the models come toward the chosen tolerance. Regardless of the step size, and this is because it will always come to 1 no matter the step size.



Section 3: Adaptive Step Size Implementation



With stiff problems like this, the solution behaves very differently in different regions: early time results in fast transient behavior, later time results in smooth oscillation around equilibrium. A fixed step size either wastes computation or becomes inaccurate. So instead, I implemented adaptive stepping.

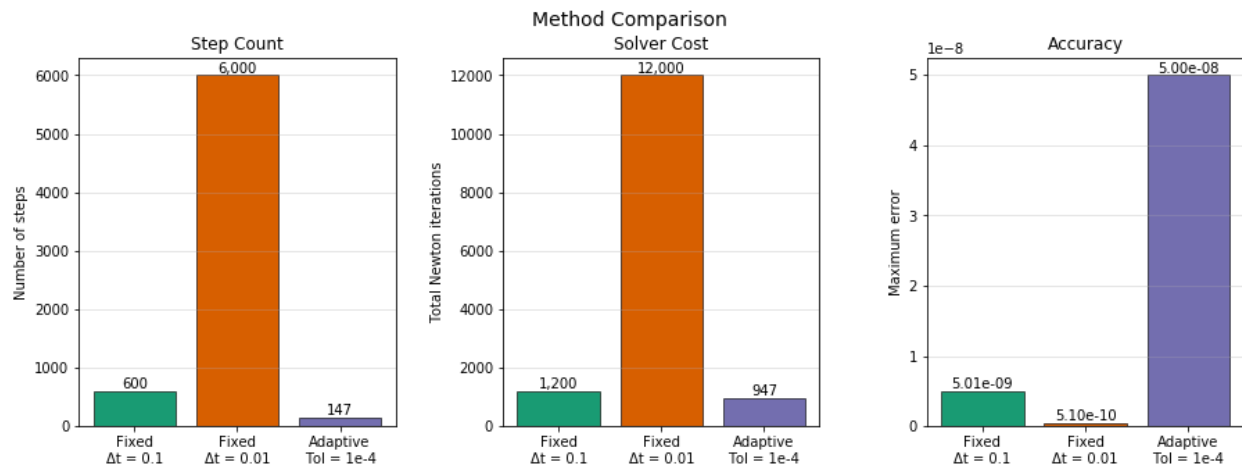
To estimate error, I used step doubling by taking one full step of size h , take two half steps of size $h/2$, and then compare the results. This works because Euler's Backward is first-order accurate, so the difference between the two solutions gives a good estimate of local truncation error.

Once I estimate the error, I update the step size using: $\text{new} = h\sqrt{\text{tol}/\text{err}}$. Then I apply safety factor, bounds on step size, and limit changes. Finally if error is less than tolerance then the step is accepted, but if not then the step is rejected and to retry with smaller h . Importantly, when a step is accepted, I use the more accurate half-step solution, not the full step result.

From Figure 3, it is shown that early in the simulation the step sizes are small, after the transient phase the step size increase very largely, and eventually the solver reaches a stable large step size near $h_{\max}$. The error plot shows that it is below tolerance majority of the time, the rejected steps occur early on, and after stabilization, almost all steps are accepted. So the solver automatically “figures out” where the problem is hard or easy.

Section 4: Performance Analysis

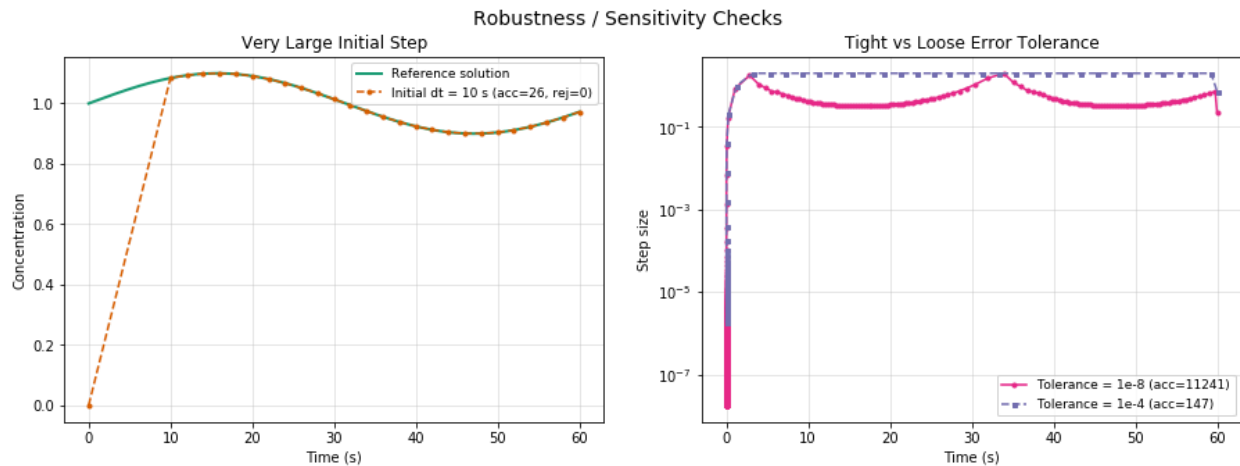
I compared three methods: fixed step $h = 0.1$, $h = 0.01$, and adaptive method.



Method	Steps	Newton Iterations	Maximum Error
Fixed (0.1)	low	moderate	Higher error
Fixed (0.01)	Very high	Very high cost	Very accurate
Adaptive	Lowest overall	balanced	Comparable accuracy

Even though adaptive stepping performs two solves per step (full + half steps), it still ends up cheaper overall because it uses large steps in smooth regions, it avoids unnecessary computation, and it reduces total number of steps significantly. So, the tradeoff is more work per step and fewer steps total.

Section 5: Robustness Testing



When I start with an extremely large step the solver initially rejected many steps, step size rapidly decreases, and once it stabilizes, it recovers and converges properly. Eventually it reaches the correct solution in a relatively small number of accepted steps. So even with a bad initial guess, the solver corrected itself automatically.

When tightening tolerance the step sizes overall become smaller, more rejections happen, and the solver becomes more conservative. Compared to a looser tolerance where accuracy improves and computation cost increases significantly. This shows that tolerance directly controls the tradeoff between accuracy and performance.

Section 6: Conclusions

In my implementation, damping is not always constantly active, but it is important as a safety feature. It becomes critical when step sizes are large, Newton guess is poor, or the system becomes nonlinear. Even though my ODE is linear, damping still prevents occasional instability and makes the solver more robust.

Adaptive step size clearly improves performance when the solution has both fast and slow parts, stiffness varies over time, and high accuracy. It allows the solver to automatically adjust instead of manually tuning step size.

From this project, I learned a lot of new and important things. Newton damping is mainly a robustness tool, and not always actively needed but important to prevent failure. Step-doubling is a simple but effective way to estimate local error. Adaptive methods really reduce computation when the system has varying behavior, and that safety factors and bounds are very much needed to prevent unstable oscillations.